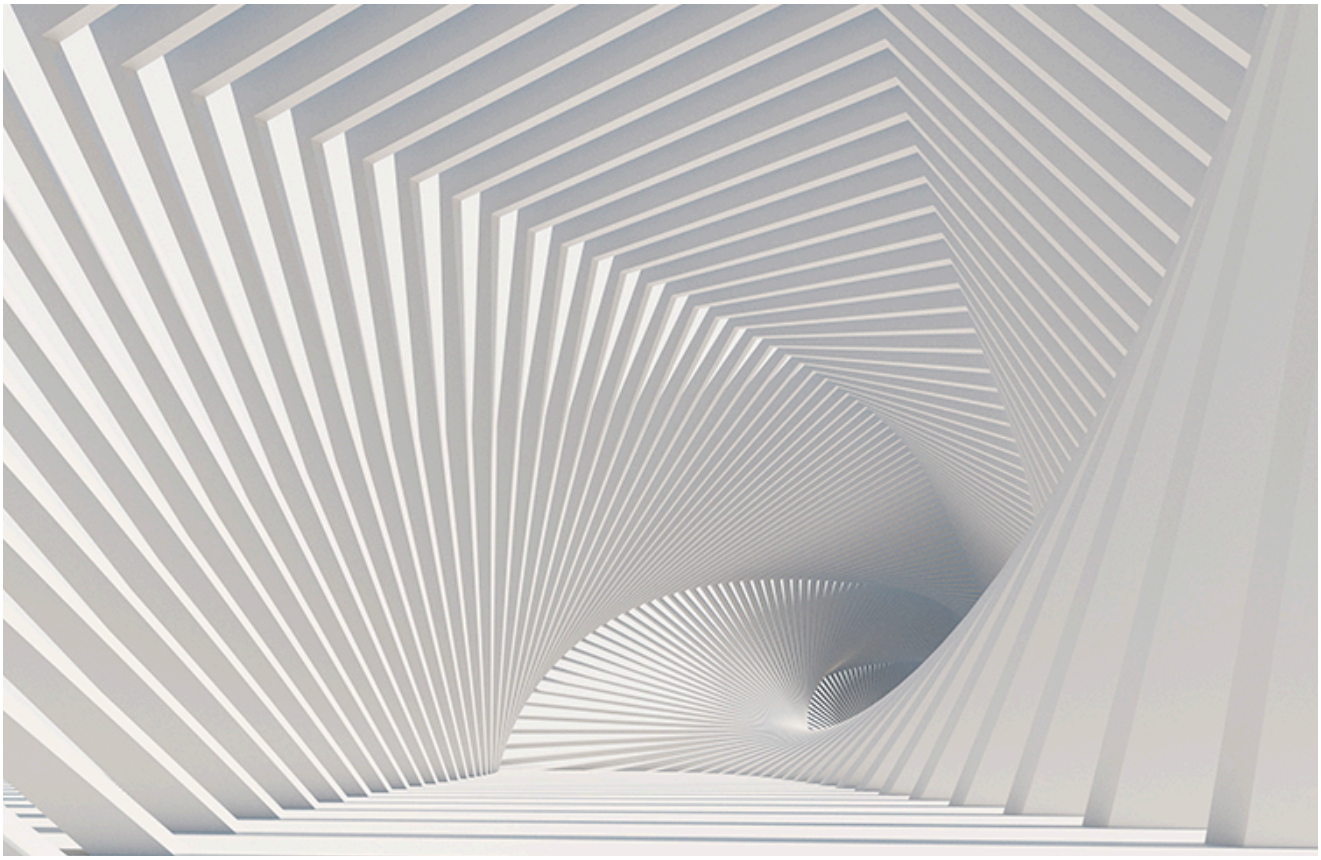
PROGRAMLAMA DİLLERİNİN PRENSİPLERİ

Proje 1 rapor



Ad Soyad : Mustafa Can Avcı
No:G2310382
Grup : 2/A

Bu ödev kapsamında, konsol ekranı üzerinden çalışan, nesne yönelimli ve dinamik bir şehir/nüfus simülasyonu geliştirmem istendi. Programın temel amacı; kullanıcıdan başlangıç tur sayısını ve ilk nüfus değerlerini alarak Şehir, İlçe, Mahalle ve Kişi hiyerarşisine tam uygun bir veri yapısı inşa etmekti. Simülasyonun ana iskeleti, turlar ilerledikçe nüfusun belirli matematiksel kurallara (birler ve onlar basamağındaki sayıların toplamı oranında) göre artırılması ve en önemlisi, nüfusu 4 basamağa (1000 ve üzerine) ulaşan şehirlerin kurallara uygun şekilde ikiye bölünerek sistemde yeni bağımsız şehirler oluşturması üzerine kuruldu.

Ödevde Yaptıklarım ve Edindiğim Tecrübeler Projenin kodlama aşamasına geçmeden önce, mimariyi Nesne Yönelimli Programlama'nın (OOP) kalbi olan Tek Sorumluluk Prensibi'ne (Single Responsibility Principle) uygun şekilde kurgulamaya özen gösterdim. Her bir sınıfın karmaşıklıktan uzak ve sadece kendi işlevinden sorumlu olması adına projeyi Main, Oyun, Sehir, Ilce, Mahalle ve Kisi sınıfları olacak şekilde modüler bir yapıda tasarladım. Bu ayrım, özellikle şehirlerin bölünmesi ve kişilerin yeni şehirlere taşınması gibi karmaşık algoritmaları kodlarken projenin okunabilirliğini ve yönetilebilirliğini büyük ölçüde artırdı.

Ödevin en önemli isterlerinden biri olan simülasyondaki kişilerin ve yerlerin (isim, soyisim, yaş vb.) sahte ama gerçekçi verilerle doldurulması işlemi için Java Faker kütüphanesini kullandım. Başlangıçta projeyi bir Maven yapısına çevirip bağımlılıkları oradan yönetmeyi denesem de, derleme aşamasında bazı konfigürasyon hataları ve uyumsuzluklar yaşadım. Simülasyonun mantıksal tarafına odaklanmak ve projeyi tehlikeye atmamak adına hızlıca alternatif bir yol izledim. Java Faker kütüphanesini bir .jar dosyası olarak manuel şekilde indirip, Eclipse ortamında projemin derleme yoluna (*Build Path*) elden dâhil ettim. Başta bir pürüz gibi görünen bu durum, aslında bana harici kütüphanelerin bir projeye manuel olarak nasıl entegre edileceğini ve bağımlılık yönetiminin arka planda nasıl çalıştığını pratik bir şekilde tecrübe etme fırsatı sundu.

Zorlanılan Kısımlar ve Karşılaşılan Engeller: Projenin genel mimarisini ve Nesne Yönelimli Programlama mantığını kurmak akıcı ilerlese de, geliştirme sürecinde bazı teknik zorluklarla karşılaştım. En çok vakit harcadığım nokta, bir turun sonunda şehrin nüfusu dört basamaklı sayılara (1000 ve üzeri) ulaştığında şehrin yarıya bölünmesi ve ilçelerin yeni şehre aktarılması algoritması oldu. İlçe sayısının tek veya çift olma durumlarına göre mahallelerin ve o mahallede yaşayan kişilerin veri kaybı yaşanmadan yeni şehre taşınması dinamiğini, kağıt üzerinde algoritmalar kurarak çözebildim.

Bağımlılık yönetimi tarafında başlangıçta Maven yapısını kullanmayı denesem de çeşitli yapılandırma hataları aldım. Ödevin teslimini riske atmamak adına, istenilen Java Faker kütüphanesinin .jar dosyasını manuel olarak indirip projenin derleme yoluna (Build Path) elden dâhil ettim. Bu sayede harici bir kütüphaneyi projeye entegre etmeyi alternatif bir yolla da olsa tecrübe etmiş oldum. Ayrıca, çalıştırılabilir JAR dosyası konsoldan başlatıldığında ekranın temizlenmesi isterini entegre ederken işletim sistemi kaynaklı ufak aksaklıklar yaşasam da, araştırma yaparak uygun çözümü koduma uyguladım.

Eksik Bırakılan Yerler ve Karşılaşılan Sorunlar: Proje dokümanında istenen temel özelliklerin (Java Faker entegrasyonu, Nesne Yönelimli Programlama prensipleri, algoritma mantığı ve satır/sütun tabanlı arama) tamamı sisteme dahil edilmiş ve arka planda hatasız çalışmaktadır. Kodun matematiksel işleyişinde bir eksiklik olmamasına rağmen, konsol arayüzünde (görsel çıktıda) bazı kozmetik eksiklikler bulunmaktadır:

- **Hizalama Sorunu:** Şehir nüfusları turlar ilerledikçe farklı basamak sayılarına ulaştığı için (örneğin aynı satırda 3 haneli bir nüfus ile 5 haneli bir nüfus yan yana geldiğinde), ekrana yazdırılan matris çıktısında köşeli parantezler [ ] alt alta jilet gibi hizalanamamış, simetrik tablo görünümü tam olarak sağlanamamıştır.
- **Okunabilirlik (Girinti) Eksikliği:** Detaylı şehir listeleme aşamasında hiyerarşik yapı (Şehir -> İlçe -> Mahalle -> Kişiler) mantıksal olarak doğru çalışmaktadır. Ancak bu hiyerarşiyi konsola yazdırırken içeriye doğru girinti (tab boşlukları) eklenmediği için, alt alta yüzlerce kişinin sıralandığı uzun listelerde görsel takip biraz zorlaşabilmektedir.
- **Konsol Temizleme Davranışı:** Her tur sonunda konsolun temizlenmesi işlemi, kullandığım terminal ortamına bağlı olarak ekranı tamamen sıfırlamak yerine bazen çok sayıda boş satır bırakarak metni aşağı kaydırma şeklinde çalışabilmektedir.